Why we moved

Tunable NM-ROM decoders, May to November 2026. Tahmid Awal, November 3, 2026.

We build reduced-order models that solve a PDE for a few numbers instead of one value per grid point. The decoder turns those few numbers into the full field, and a Gauss-Newton solver adjusts them until the field satisfies the PDE. We need two things from a decoder: the solve has to be accurate, and each solver iteration has to be cheap, or there is no point in using a reduced model.

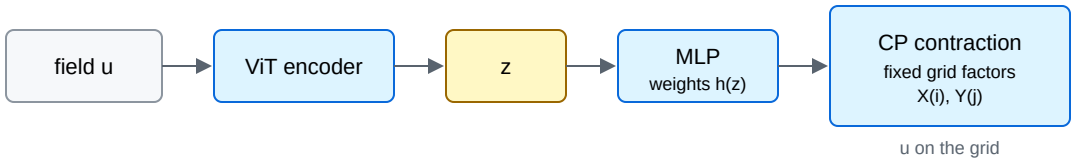
We went through five decoder designs. The short version:

Period	Decoder	Why we left it
May–Jul	ViT encoder + CP / LinearCP	Worked on heat and Poisson, but the rollout diverged on Burgers 2D
May	ViT decoder	Slower than the full solver (0.00–0.02×), and the solves failed
May	SIREN coordinate decoder	Too slow: 3–8× at CP-level accuracy, against 183× for CP
Aug	FiLM coordinate decoder	Too slow: the network runs inside every solver iteration
Aug–now	Learned bank (ours)	Current method

Speedup means full-solver time divided by reduced-model time, so 2× is twice as fast and 0.5× is half as fast. Error is relative L2 error unless noted. Each section links to the code; the links open a copy of each file pinned at commit `dd6c6c0` in the project repository.

---

1. ViT encoder + CP / LinearCP decoder (May–July)



$$u(i, j; z) = b + \Sigma_a \, h_a(z) \, X_a(i) \, Y_a(j)$$

WHY WE TRIED IT

This was the original method. A ViT encodes a field to a latent z, a small MLP turns z into weights, and the weights multiply fixed tables of grid factors. The value at a single grid node is cheap to evaluate, so the solver only touches the few nodes picked by empirical quadrature (EQ). LinearCP adds a linear skip from z so that Poisson can start the solve from z = 0.

WHAT HAPPENED

Case	Error	Speedup	Note
Poisson 3D, N=64 (LinearCP)	6.62e-3	111×	published table
Heat 3D, N=64 (CP)	1.76e-2	269×	published table
Heat 2D, N=64, fixed package	2.83e-2	0.17×	after fixing a rollout bug in the clean package; published was 5.21e-3 at 39.6×
Burgers 2D, ViT-CP	diverges	–	rollout blows up
Burgers 2D, retrained on more data	5.7e-3 recon	–	reconstruction 3.6× better, rollout still diverges
Burgers 2D, frozen POD instead	2.1e-2	3.9×	what we used for Burgers

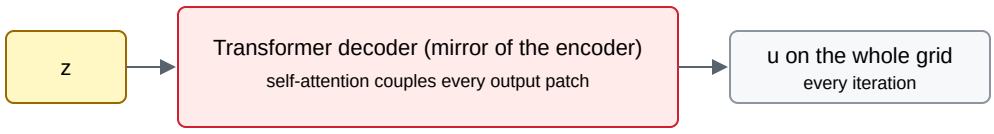
WHY WE MOVED ON

We could not get it to work on Burgers. Better reconstruction did not help: the 2D Burgers rollout diverged even after retraining improved reconstruction 3.6×, so the Burgers results came from a frozen POD basis rather than the learned decoder. The grid factors are also tied to one mesh, and accuracy got worse as the mesh got finer. We needed a different decoder.

cp/decoder.py CP decoder (heat) · linearcp/decoder.py LinearCP (Poisson) · old_podmlp/podmlp.py POD + MLP · pod_hr/hrcore.py POD with hyper-reduction
Results: July results README · July results ledger

---

2. ViT decoder (May)



WHY WE TRIED IT

A ViT encoder was already working, so the obvious test was a matching ViT decoder in place of CP, as a more expressive output map.

WHAT HAPPENED

Case	Autoencoder error	ROM error	Speedup
Poisson 3D N=64, LinearCP baseline	6.62e-3	6.62e-3	111×
Poisson 3D N=64, ViT decoder (best of 3 sizes)	5.84e-3	1.25	0.00×
Heat 3D N=64, CP baseline	1.76e-2	1.76e-2	269×
Heat 3D N=64, ViT decoder (3 sizes)	3.1e-2 – 3.8e-2	8.6 – 13.0	0.01–0.02×

WHY WE MOVED ON

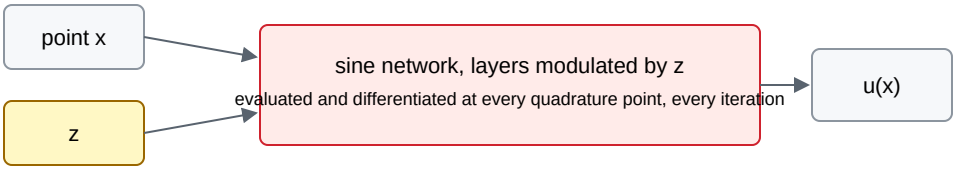
It was too slow to be useful. Self-attention couples all outputs, so the solver has to run the whole decoder on all 262,144 grid nodes in every Gauss-Newton iteration, and the quadrature saves nothing. Every configuration was slower

than the full solver. The solves also failed, even when the autoencoder was as good as LinearCP.

vit/decoder.py mirrored ViT decoder · xattn/cross_attn_inr.py cross-attention variant
Results: ViT ablation status report

---

3. SIREN coordinate decoder (May)



$$u(x; z) = f_\theta(x, z)$$

WHY WE TRIED IT

A coordinate network is not tied to a grid. It can be evaluated at any single point, so it fits the quadrature idea, and its spatial features can change with z, which CP's cannot.

WHAT HAPPENED

Poisson 2D, N=128, against a CG full solver:

Decoder	Median error	Speedup	Time per solve
CP decoder (reference)	3.23e-3	183×	–
SIREN, first working version (warm start)	5.12e-2	1.0×	–
SIREN after training fixes (warm start)	2.98e-3	8.4×	–
SIREN baseline solver (cold start)	2.88e-3	0.13×	2152 ms
SIREN with faster Jacobians, tuned	2.88e-3	3.44×	80 ms
Linear-affine SIREN, u = Φ(x)Az + b(x)	9.81e-3	139×	1.97 ms

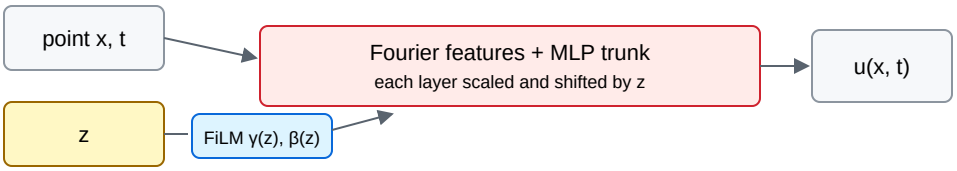
Cold-start solves also stalled in a spurious basin near z = 0 until the training was changed.

WHY WE MOVED ON

It was too slow for our use. At CP-level accuracy the best SIREN was 3–8× faster than the full solver, against 183× for CP, because the whole network has to be evaluated and differentiated at every quadrature point in every iteration. The only fast version was the one where the output is linear in z and the spatial network Φ(x) is fixed, at the cost of about 3× more error. That observation is what later became the learned bank.

siren/modulated_siren.py · default_siren/default_field_siren.py · skip_siren/linear_skip_siren.py · affine/affine_z.py · linear_affine/linear_affine_z.py
Results: SIREN speed/accuracy summary · SIREN training history

4. FiLM coordinate decoder (August)



u(x; z) = MLP(features(x)) with every layer scaled and shifted by functions of z

WHY WE TRIED IT

FiLM represented the families much better than a plain coordinate MLP and transferred across meshes. With latent stepping (solving for z from the PDE at each time step) it became a real reduced model for Poisson, heat, Burgers and wave.

WHAT HAPPENED

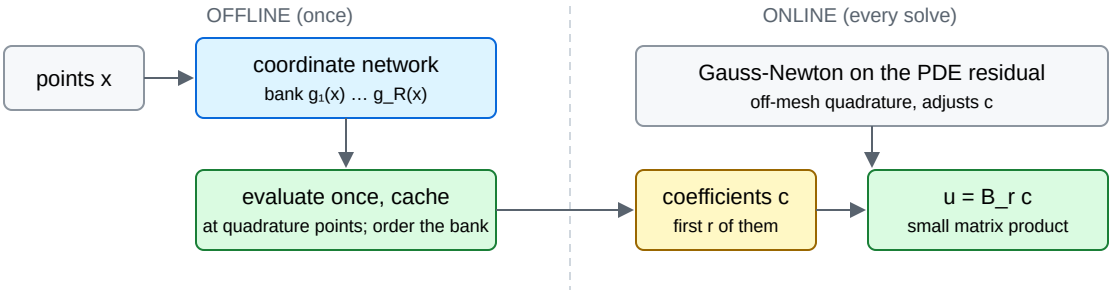
Case	Result
Grouped FiLM, Poisson 2D (58,419 parameters)	about 0.5% mean solve error over 3 seeds
Grouped FiLM, Burgers 2D (140,449 parameters)	about 1.0% mean solve error over 3 seeds
Latent-size sweep, k = 4 to 64, Poisson and Burgers	error stops improving near 1e-2 by k = 16–24, while cost grows about 170× across the sweep
Burgers, FiLM k=16 vs learned bank k=16	about 507 Jacobians for FiLM vs about 315 for the bank
Poisson, grouped FiLM warm start	a plain spectral solver was tens of milliseconds faster

WHY WE MOVED ON

Speed. Accuracy was fine, but the full network still runs and is differentiated inside every solver iteration, and cost rose steeply with latent size. The fix was to take the spatial network out of the solver loop.

film/poisson2d_film.py · film_heat/heat2d_film.py · film_burgers/burgers2d_film.py · film_wave/wave2d_film.py · latent_burgers/blat_common.py latent stepping · film_arch/nda_arch.py grouped FiLM
Results: grouped FiLM numbers (JSON)

5. Learned bank: our current decoder (August–now)



$$u(x; c) = \Sigma_{a \le r}\, g_a(x)\, c_a = B_r(x)\, c$$

WHY WE BUILT IT

This is the fast SIREN variant taken further. A coordinate network learns a set (bank) of spatial functions $g_a(x)$. We evaluate the bank once at the quadrature points and cache it. Online, the solver only works with the cached matrix and a few coefficients, so no neural network runs inside the solver loop. At first a small nonlinear head mapped a latent z to the coefficients. Later we dropped the head and solve for the first r bank coefficients directly, because a head can never produce a field outside the bank's span. The number r is a knob we can set at deployment time to trade accuracy for cost.

WHAT HAPPENED

First version (August), Burgers 2D N=64, same GPU and job:

Model	Rollout time	Error (4 test trajectories)
Full solver (Newton)	176.1 ms	reference
Learned bank, K=16, cached	54.1 ms (3.3× faster)	1.38e-2, 2.10e-2, 2.58e-2, one outlier 1.47e-1

Current version (October campaign), Burgers 2D at 1024^2, 38 test cases, second-order stepping:

Bank width r	Median error	Worst error
128	0.45%	5.2%
256	0.209%	2.52%
384	0.198%	1.78%
512	0.199%	2.17%

Question	Result
Does solve cost grow with the mesh?	No. 2D flat from 256^2 to 4096^2 (ratio 1.01–1.02); 3D flat from 64^3 to 384^3 (1.01–1.09)
Does accuracy depend on the mesh?	No, within the set margin, in all 8 settings tested
Burgers 3D, coarse to fine bank setting	median error 2.04% → 0.18%, cost 542 → 2063 ms
Faster than the best tuned full solver?	No. A coarse spectral solver reaches the same accuracy faster: our speedup 0.049–0.71× in 2D, 0.008–0.16× in 3D
Better than tuned POD with hyper-reduction?	No. Tuned POD did better on all 8 tested 2D settings at 256^2 and at 1024^2

WHERE THIS LEAVES US

The bank is the first design that is both accurate and cheap per iteration, and its cost does not grow with the mesh. On Burgers in a box it does not beat a tuned spectral solver or tuned POD, so we cannot claim a speed advantage there. What we can claim is mesh-independent cost, evaluation at any point, and lower memory. That is why the current work moved to flow past obstacles, where spectral solvers do not apply. The first obstacle bank had 18–21% velocity error; after retraining, the latest steady solves pass all seven development targets with about 0.036% velocity error. Final validation and timing are still to do, and the unsteady cylinder case (Re ≈ 100) started on November 3.

separable/sep_common.py first bank + head · current_head/hcore.py bank + head, current · correction/correction_core.py correction directions · ordered/qcore.py ordered bank + off-mesh quadrature · current_step/t2core.py second-order stepping · wide/w3d.py wide bank, 3D · curlbank/coordnet.py divergence-free bank (NS) · ns-obstacle-rom/ obstacle-flow ROM
Results: Burgers campaign final report · Burgers 3D refinement report · bank design note

Other things we tried on top of the bank

These kept the bank and changed something else. None of them became the main method.

Variant	Result	Code
Bigger/deeper MLP heads	Lower training error, no better solves	sep_hfit.py
Quadratic head	Mean 4.7% vs 5.5% for the MLP, worst 20.3% vs 17.0%	b3d_arch_quadratic.py
Protected-anchor head	Mean 6.1%, worst 17.7%; failed	b3d_arch_anchor.py
Shared encoder for the head	Mean 5.4%, worst 16.2%; not consistent across repeats	b3d_arch_encoder.py
Mixture of local heads	Mean 5.5%, worst 21.3%; no gain	b3d_arch_mixture.py
Wider and smoother banks	Better fits, not better solves	w3d.py, smoothtrain.py
Moving frame (NS, translating flows)	Worked for that family; a nonlinear head helped there	shift_rom.py, head_rom.py
Coarse-to-fine starts (prefix, separate decoder, shared decoder)	Slower than solving directly: 1.39–3.38×	prefixcore.py, transfer.py

Head numbers are fitted validation errors on Burgers 3D with the same frozen bank (optimizer seed 200).

What we took from this

- Good reconstruction did not mean a good solve. ViT-CP on Burgers and the ViT decoder both reconstructed well and failed in the solver.
- Anything that runs a neural network inside the solver loop was too slow (ViT, SIREN, FiLM). Caching the spatial functions is what fixed speed.
- A nonlinear head cannot leave its bank's span, so most of the head work did not pay off.
- Tuned classical baselines (spectral solvers, POD) decide whether a result is a real win, so we compare against them at matched accuracy.

Sources: decoder-history report (Oct 28), ViT ablation status (May 14), SIREN speed summary (May 18), July results README and ledger, project lab log, Burgers campaign final report (Oct 20), Burgers 3D refinement report (Oct 27). Numbers are copied from those records.